Introduction to Computer Science and Programming

Loops and Functions in EduBlocks

Laboratory 3, October 21, 2024

Filippo Ricca

DIBRIS, University of Genova

Repetition

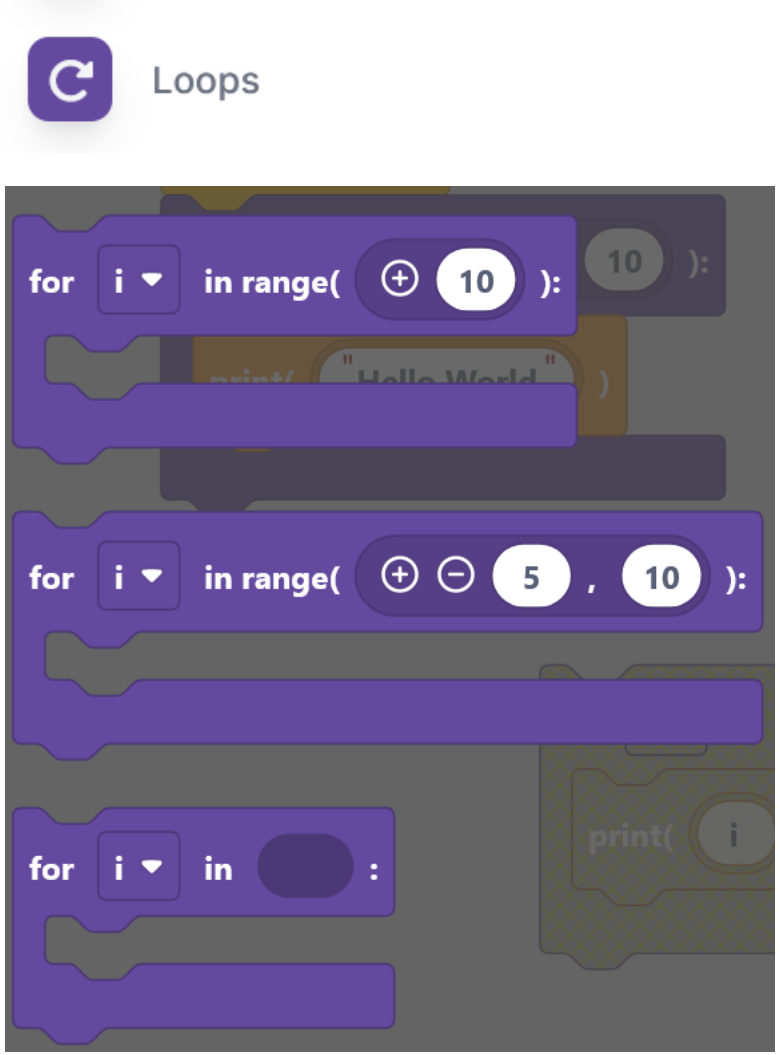
- Doing something ‘over and over’ again
- Used in combination with decision making
  - Otherwise, we loop forever
    - This is called an “infinite loop”
- The **Repetition structure** can be implemented using different commands:
  - The While Loop
  - The For Loop

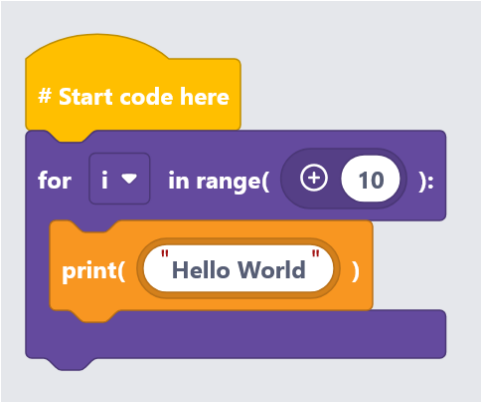
WHILE condition p is true
Statement(s) to execute
ENDWHILE

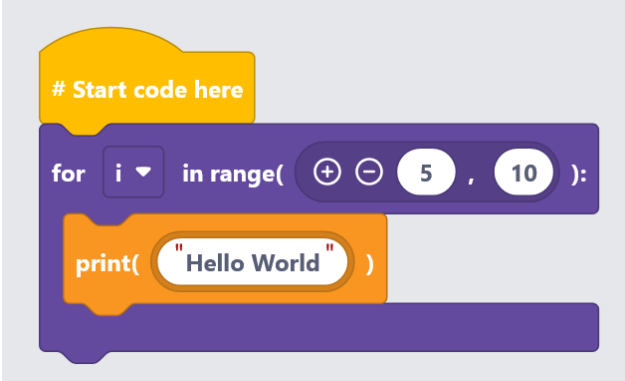
FOR a number of times
Statement(s) to execute
ENDFOR

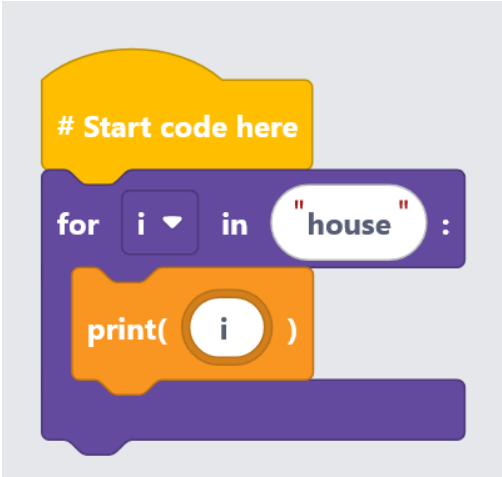
Loops section

For Loop



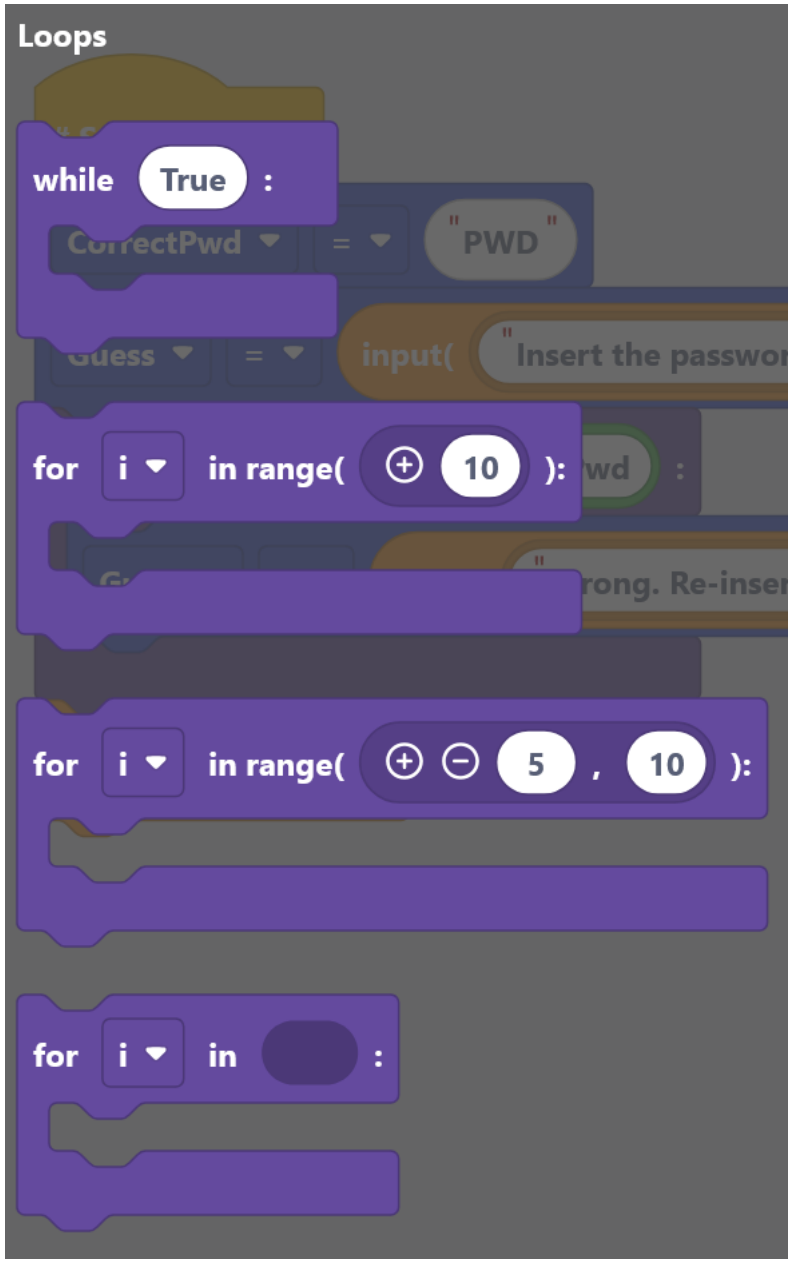




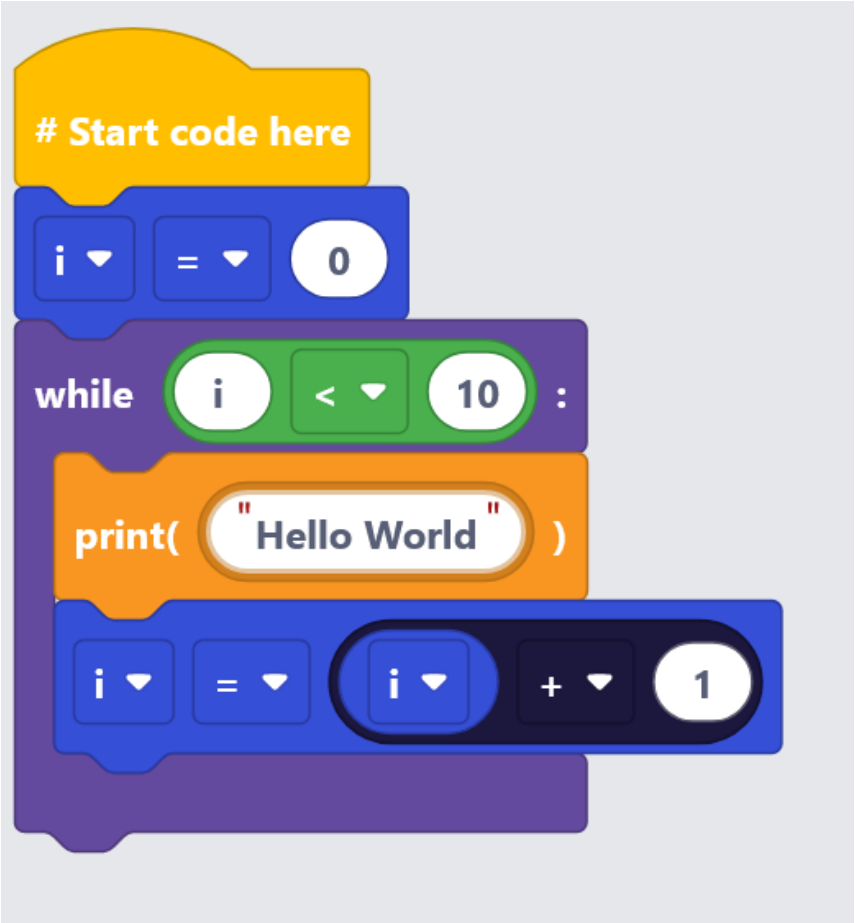


‘H’ ‘o’ ‘u’ ‘s’ ‘e’

Loops



While Loop

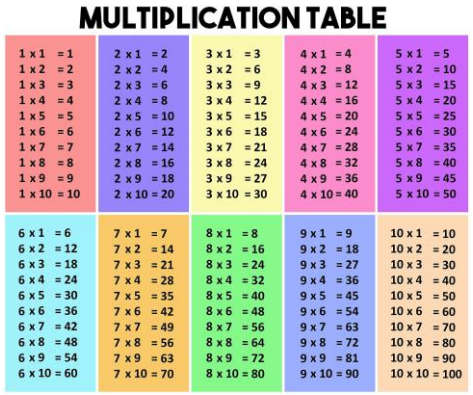


Exercise 1: Multiplication Tables (“Le Tabelline”)

Problem Statement:

- Write an **EduBlocks program** that:
  - Asks the user which number’s multiplication table (“tabellina”) they want to print
  - Prints the table of that number from 1 to 10
  - For example, if the user enters 3, the program prints:

3
6
9
12
…
30



MULTIPLICATION TABLE

1 x 1 = 1	2 x 1 = 2	3 x 1 = 3	4 x 1 = 4	5 x 1 = 5
1 x 2 = 2	2 x 2 = 4	3 x 2 = 6	4 x 2 = 8	5 x 2 = 10
1 x 3 = 3	2 x 3 = 6	3 x 3 = 9	4 x 3 = 12	5 x 3 = 15
1 x 4 = 4	2 x 4 = 8	3 x 4 = 12	4 x 4 = 16	5 x 4 = 20
1 x 5 = 5	2 x 5 = 10	3 x 5 = 15	4 x 5 = 20	5 x 5 = 25
1 x 6 = 6	2 x 6 = 12	3 x 6 = 18	4 x 6 = 24	5 x 6 = 30
1 x 7 = 7	2 x 7 = 14	3 x 7 = 21	4 x 7 = 28	5 x 7 = 35
1 x 8 = 8	2 x 8 = 16	3 x 8 = 24	4 x 8 = 32	5 x 8 = 40
1 x 9 = 9	2 x 9 = 18	3 x 9 = 27	4 x 9 = 36	5 x 9 = 45
1 x 10 = 10	2 x 10 = 20	3 x 10 = 30	4 x 10 = 40	5 x 10 = 50
6 x 1 = 6	7 x 1 = 7	8 x 1 = 8	9 x 1 = 9	10 x 1 = 10
6 x 2 = 12	7 x 2 = 14	8 x 2 = 16	9 x 2 = 18	10 x 2 = 20
6 x 3 = 18	7 x 3 = 21	8 x 3 = 24	9 x 3 = 27	10 x 3 = 30
6 x 4 = 24	7 x 4 = 28	8 x 4 = 32	9 x 4 = 36	10 x 4 = 40
6 x 5 = 30	7 x 5 = 35	8 x 5 = 40	9 x 5 = 45	10 x 5 = 50
6 x 6 = 36	7 x 6 = 42	8 x 6 = 48	9 x 6 = 54	10 x 6 = 60
6 x 7 = 42	7 x 7 = 49	8 x 7 = 56	9 x 7 = 63	10 x 7 = 70
6 x 8 = 48	7 x 8 = 56	8 x 8 = 64	9 x 8 = 72	10 x 8 = 80
6 x 9 = 54	7 x 9 = 63	8 x 9 = 72	9 x 9 = 81	10 x 9 = 90
6 x 10 = 60	7 x 10 = 70	8 x 10 = 80	9 x 10 = 90	10 x 10 = 100

Pseudocode

Version 1 (abstract pseudocode)

"Ask the user which number's table to print"

"For each number from 1 to 10 multiply by the chosen number"

"Print only the result of each multiplication"

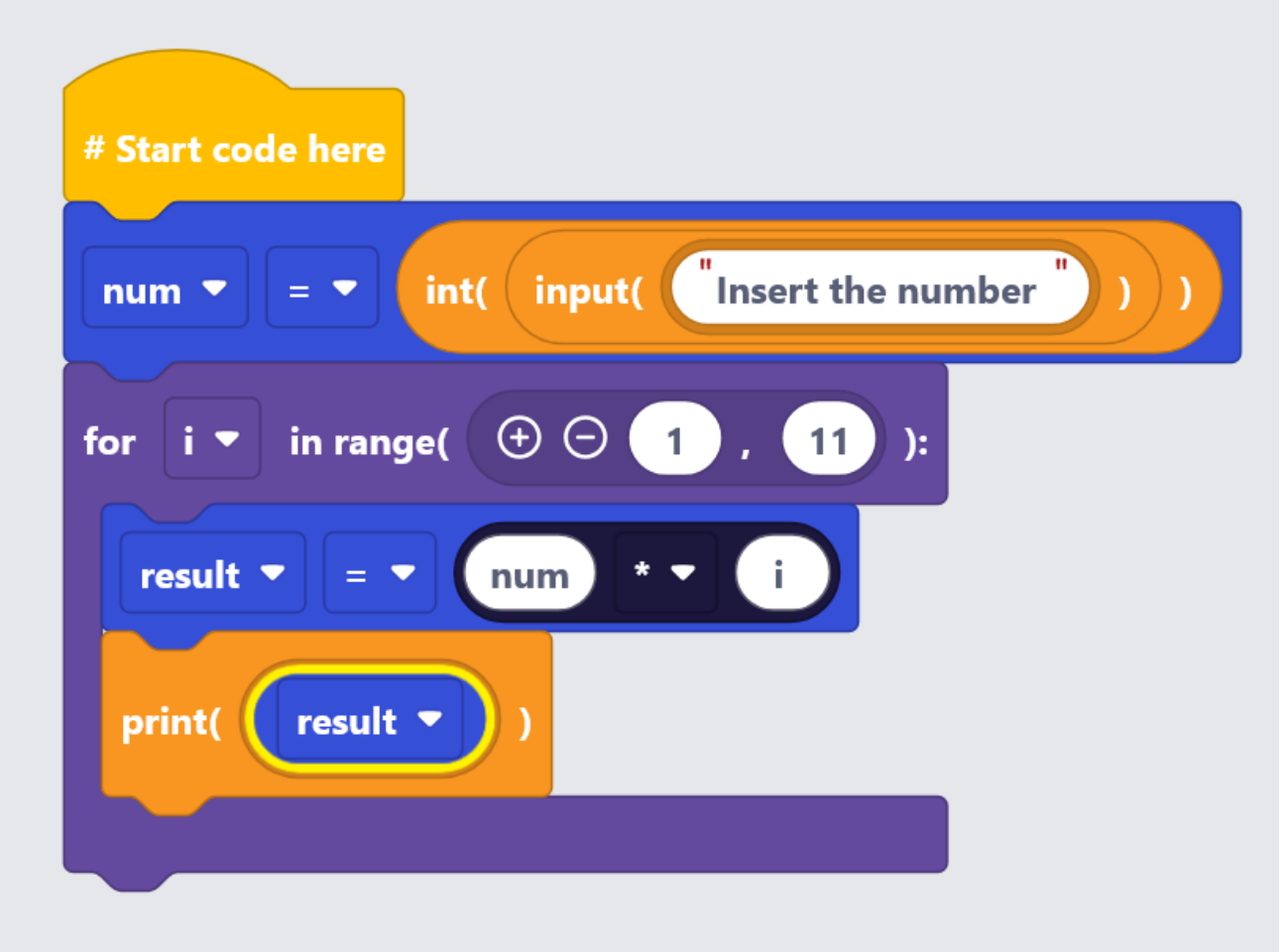
Version 2 (refinement)

```
GET num
FOR i ← 1 TO 10 DO
   result ← num × i
   DISPLAY result
ENDFOR
```

EduBlocks Implementation: Multiplication Table

Version 2 (refinement)

```
GET num
FOR i ← 1 TO 10 DO
    result ← num × i
    DISPLAY result
ENDFOR
```



Exercise 2: Password

Problem Statement:

- Write an **EduBlocks program** that:
  - Asks the user for a password and keeps asking until they enter the correct one

EduBlocks Implementation: Password

Version 1 (abstract pseudocode)

"Set the correct password."

"Ask the user to enter the password."

"While the password entered is not correct, ask again."

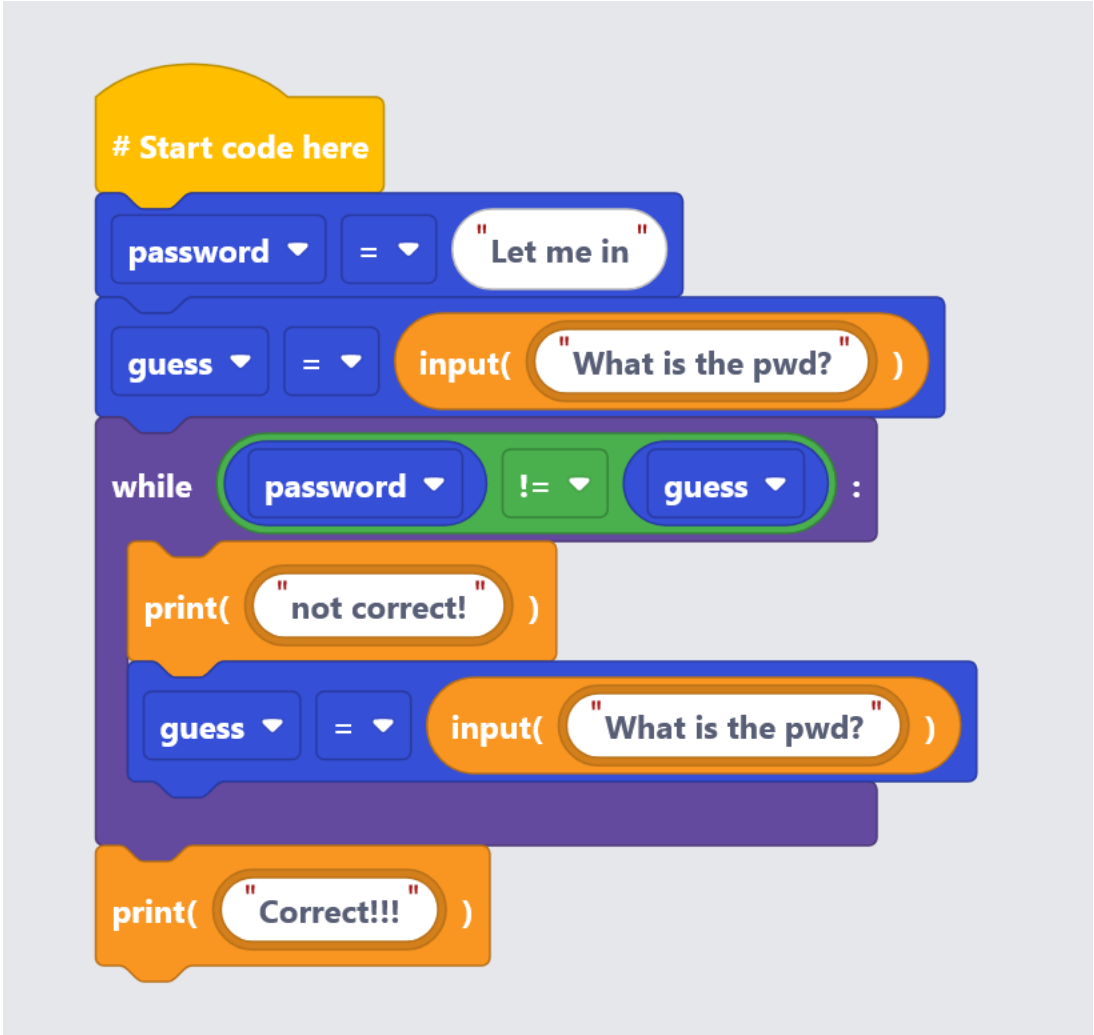
"When the password is correct, display a confirmation message."

Version 2 (refinement)

```
SET CorrectPassword ← "Let Me In"
GET guess ← INPUT("What is the Password?")

WHILE guess ≠ CorrectPassword DO
    GET guess ← INPUT("Wrong! What is the Password?")
ENDWHILE

DISPLAY "Correct!"
```



Break

```
# Start code here
counter = 0
while True :
    counter = counter + 1
    print( counter )
print( "The end" )
```

The program runs forever

The program goes into an infinite loop

The program never terminates

```
# Start code here
counter = 0
while True :
    if counter == 10 :
        break
    print( counter )
    counter = counter + 1
print( "The end" )
```

The program halts when counter is 10

How does a Function work?

We are going to **create a function** that computes the max among three integer numbers

- Go to **Definitions** and drag '**def block**' into the coding area
- In the first blank, type **Max** this will define the **name of the function**

We have now created a function. But we need to add the parameters and the code to the function. This is the code that will be ran when the function is called

COMPUTE_MAX(A, B, C):

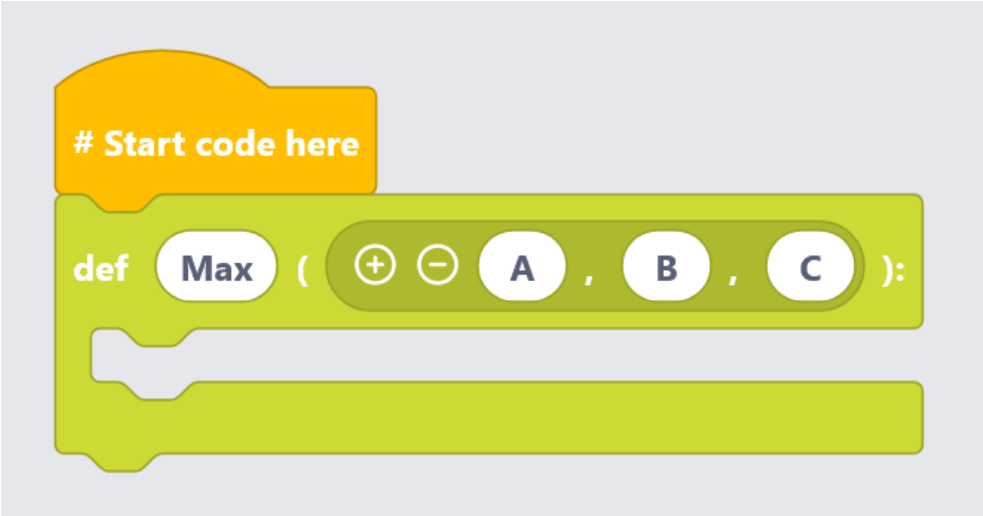
1. **IF** A>B **THEN** BIG <- A
   ELSE BIG <-B;
2. **IF** C>BIG **THEN** BIG <- C;
3. **RETURN** BIG

Functions with parameters

We are going to add parameters to the function **Max**

- Add the parameter ‘A’ in the blank space
- Click on ‘+’ and add the second parameter ‘B’
- Click on ‘+’ and add the third parameter ‘B’
- The name of the function is **Max** and it has an three parameters (A, B and C). Parameters are also called **arguments**

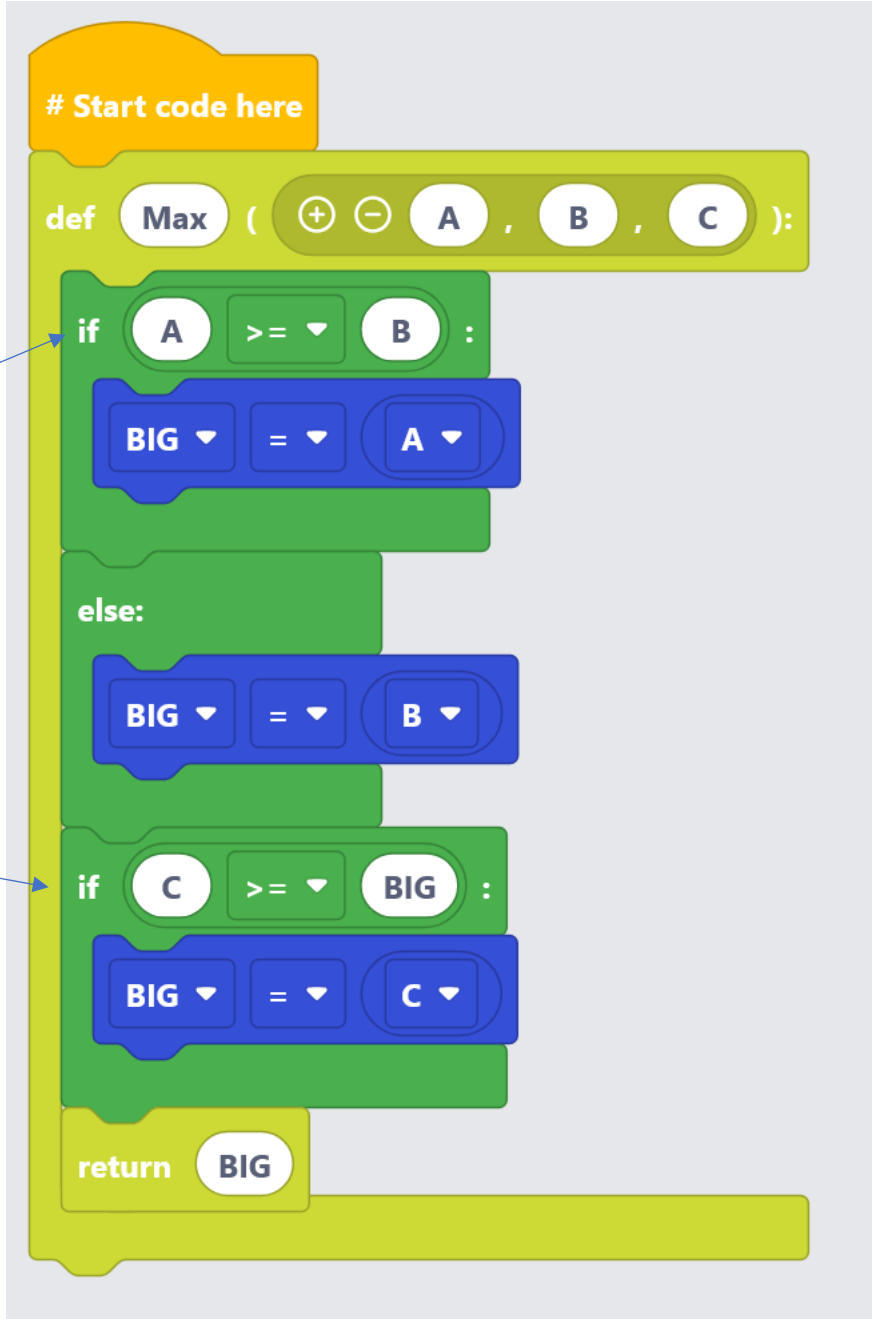
We have now created a function with three parameters



Body of the Function

Inside of the function we have to add the code we want to execute when we call it. In particular:

- If then else command
- If then command



Using a Function

Since the function return a value, we can use Print to display it, **placing inside the function call with the actual values** we want to pass to it

```
# Start code here
def Max ( A , B , C ):
    if A >= B :
        BIG = A
    else:
        BIG = B
    if C >= BIG :
        BIG = C
    return BIG
print( Max(3, 8, 9) )
```

Function call inside Print command

Why Functions are useful?

Functions are powerful tools in programming

They are small subroutines — short sequences of code that can be defined inside the main program

When we *call* a function, the program temporarily leaves the main code, **executes the instructions inside the function**, and then returns to continue from where it left off

Functions make our code **reusable**, **organized**, and **easier to read**, while also reducing the number of lines we need to write

Complete Algorithm

Final Version:

1. **GET** A, B, C;
2. BIG <- COMPUTE_MAX(A, B, C);
3. SMALL <- COMPUTE_MIN(A, B, C);
4. RESULT <- BIG – SMALL;
5. **DISPLAY** RESULT

COMPUTE_MAX(A, B, C):

1. **IF** A>B **THEN** BIG <- A
   ELSE BIG <-B;
2. **IF** C>BIG **THEN** BIG <- C;
3. **RETURN** BIG

COMPUTE_MIN(A, B, C):

1. **IF** A<B **THEN** SMALL <- A
   ELSE SMALL <-B;
2. **IF** C<SMALL **THEN** SMALL <- C;
3. **RETURN** SMALL

Exercise 3: Calculate the difference between the largest and the smallest

Problem Statement: Write an **EduBlocks** program that given three integers, calculate the difference between the **largest** and the **smallest.** Follow the pseudocode given in the slide

Input/Output:

- 6, **2**, **9** -> 7

Complete Algorithm

Final Version:

1. **GET** A, B, C;
2. BIG <- COMPUTE_MAX(A, B, C);
3. SMALL <- COMPUTE_MIN(A, B, C);
4. RESULT <- BIG – SMALL;
5. **DISPLAY** RESULT

COMPUTE_MAX(A, B, C):

1. **IF** A>B **THEN** BIG <- A
   ELSE BIG <-B;
2. **IF** C>BIG **THEN** BIG <- C;
3. **RETURN** BIG

COMPUTE_MIN(A, B, C):

1. **IF** A<B **THEN** SMALL <- A
   ELSE SMALL <-B;
2. **IF** C<SMALL **THEN** SMALL <- C;
3. **RETURN** SMALL

Exercise 4: Factorial

Problem Statement:

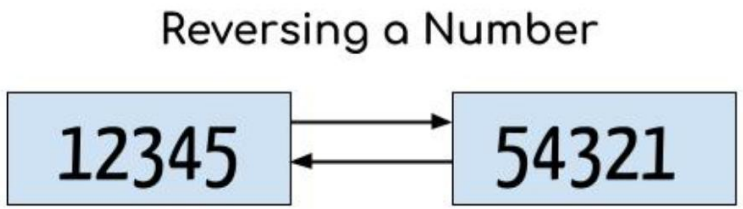
- Create a **Flowchart** for a program that calculates the factorial of a given positive integer. The factorial of a number n (denoted as n!) is the product of all positive integers less than or equal to n. Use Draw.io tool to represent the flowchart
  - For example, 5!=5×4×3×2×1=120

$$
\begin{aligned}
0! &:= 1 \\
1! &= 1 & &= 1 \\
2! &= 1 \cdot 2 & &= 2 \\
3! &= 1 \cdot 2 \cdot 3 & &= 6 \\
4! &= 1 \cdot 2 \cdot 3 \cdot 4 & &= 24 \\
5! &= 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 & &= 120 \\
6! &= 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 & &= 720 \\
7! &= 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 & &= 5'040 \\
8! &= 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \cdot 8 & &= 40'320 \\
9! &= 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \cdot 8 \cdot 9 & &= 362'880
\end{aligned}
$$

Exercise 5: Reverse String

Problem Statement: Reverse a String Representation of a Number

- Write a pseudo-code and a **EduBlock** program that reads a string representation of an integer from the user and reverses the string. The program should handle both positive and negative numbers as strings

- **Input:** A string STR_NUM representing an integer
  - it can be positive or negative

- **Output:** The reversed string representation of the number